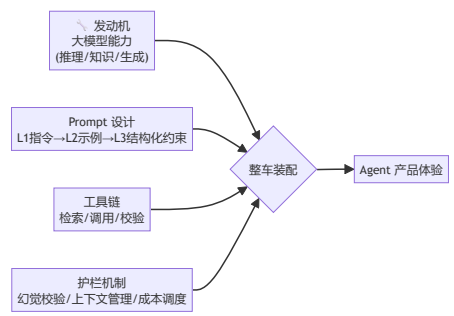


03 大模型是发动机，不是车

一个容易被忽略的比喻

很多产品经理选模型的方式，是打开一张跑分榜，看谁的分数高就选谁——这就像买车只看发动机马力参数，不看这台车到底适合拉货还是家用。**大模型是Agent 的发动机，不是整车。** 发动机参数很重要，但决定产品体验的，是发动机和整车（你的 Prompt 设计、工具链、护栏机制）匹配得好不好。

这一章不讲 Transformer 的数学公式（网上有的是），只讲产品经理必须建立的三层"发动机认知"。



第一层认知：模型在做什么——本质是"续写"，不是"理解"

不管多先进的大模型，它训练时做的核心任务就是一件事：**给定前面的文字，预测下一个最可能的词**。这个机制叫自回归生成，背后靠的是 Transformer 的注意力机制——简单说，模型在生成每一个词的时候，会"回头看"前面所有的词，判断哪些词跟当前最相关，然后综合这些信息做预测。

这个机制带来两个产品级的重要推论：

- **它没有"真正理解"这回事，它只是在做极其高级的统计续写。**这解释了为什么模型会"一本正经地编造"（幻觉）——它只是在续写一个"看起来很像对的答案"的文字序列，不代表它内部有一个"事实核查"机制。
- **它对"离题"很敏感。**如果你的 Prompt 里混入了无关信息，模型的注意力会被分散，续写质量会明显下降——这直接决定了 Prompt 工程和上下文工程的重要性（我们会在 Part 3 详细展开）。

第二层认知： Prompt 工程的三个层次

层次	做法	适用场景
L1 指令	直接说清楚要做什么、输出什么格式	简单确定性任务
L2 示例 (Few-shot)	给1-3个"输入→输出"的样例，让模型模仿	输出格式复杂、需要统一风格
L3 结构化约束	用 JSON Schema /函数签名强制约束输出结构	Agent 需要稳定解析模型输出去调用工具时 (几乎是刚需)

产品经理最容易踩的坑：只做了 L1，就指望模型在 Agent 场景里稳定输出可解析的结构。只要你的 Agent 需要用代码解析模型输出去决定下一步动作，就必须上 L3，否则解析失败率会让你的"自主决策循环"变成"随机崩溃循环"。

第三层认知： 2026 年主流模型 全景图（产品经理 版）

跑分榜会一直变，但"按场景选模型"的框架是稳定的。这是一份按产品维度（而非跑分）划分的参考表：

维度	该选什么类型的模型	2026 年代表（示例，具体以官网价目为准）
强推理 / 复杂规划	支持"深度思考"模式的推理增强模型	DeepSeek-V3.2 深度思考模式、混元最新推理模型
高并发 / 低成本路由	参数量小、延迟低、按量计费便宜的模型	各家的 flash/turbo/lite 系列
长文本 / 知识库问答	长上下文窗口 + 检索增强能力强	支持128k+上下文的旗舰模型
多模态	原生多模	各家最新多模态旗舰

维度	该选什么类型的模型	2026 年代表 (示例, 具体以官网价目为准)
(图文/语音)	态而非"外挂 OCR"	
国内合规部署	有明确的企业级 SLA 和合规资质	云厂商自有模型 (如混元) + 国产开源模型自部署

给产品经理的选型建议：不要为整个 Agent 系统只选一个模型。

一个成熟的 Agent 产品往往是"分层调度"——用便宜快的小模型做意图分类/路由，用强模型做真正需要推理的那一步，这是控制成本的关键手段（后面案例复盘会具体展开这个架构）。

三大局限，直接决定 Agent 设计

局限	具体表现	对 Agent 设计的直接倒逼
幻觉	会自信地编造不存在的事实/API/数据	必须设计"结果校验环节"——比如让工具调用返回真实数据而不是让模型编，

局限	具体表现	对 Agent 设计的直接倒逼
		关键结论要有可追溯的信息来源
上下文窗口有限	对话/任务变长后，前面的信息会被"挤出去"或被稀释	必须设计记忆架构（短期上下文+长期检索），不能假设模型"记得住"
成本与延迟	强模型贵且慢，不可能每一步都调用最强模型	必须做分层调度：路由用小模型，推理用大模型，缓存能复用的结果

产品经理清单

- ☐ 我选模型的依据，是场景需求（长文本/强推理/低成本），还是单纯的跑分榜排名？
- ☐ 我的 Agent 里，有没有一个环节是"直接相信模型编出来的事实"，而没有做校验？如果有，这是幻觉风险最高的地方，需要优先加护栏。
- ☐ 我的架构里是不是每一步都在用最贵的模型？有没有可以用小模型做路由/

分类，把大模型留给真正需要推理的步骤？

下一章，我们把这些认知汇总成一个可以直接拿去用的框架——**Agent 产品成熟度分级 L0~L3**，帮你判断自己的产品到底该造到哪一级。

章节自测（5道单选，答案见文末）

Q1. 大模型训练时做的核心任务本质是什么？ A. 理解语义 B. 给定前文预测下一个最可能的词（自回归续写） C. 检索知识库 D. 执行代码

Q2. Prompt 工程的 L3"结构化约束"最适合什么场景？ A. 简单确定性任务 B. Agent 需要稳定解析模型输出去调用工具 C. 闲聊场景 D. 不需要输出格式的场景

Q3. 产品经理选模型最容易踩的坑是什么？ A. 选太贵的模型 B. 只看跑分榜排名而不考虑场景需求 C. 选太便宜的模型 D. 不看文档

Q4. 文中给出的应对"幻觉"风险的设计倒逼是什么？ A. 换更贵的模型 B. 设计结果校验环节，让工具调用返回真实数据而不是让模型编 C. 增加训练数据 D. 缩短 Prompt

Q5. 一个成熟 Agent 产品的模型调度建议是什么？ A. 全部用最强模

型 B. 全部用最便宜模型 C. 分层调度：小模型做路由/分类，强模型做真正需要推理的步骤 D. 随机选择

► [点击查看参考答案](#)